

	IT-Technik cyber-physische Systeme		Architektur Espressif IDF	
	Nachname, Vorname:		Klasse: IF11	Datum:

1. Die typische Verzeichnisstruktur

Ein minimales, lauffähiges ESP-IDF Projekt sieht auf Dateiebene so aus:

```

mein_esp_projekt/
├── CMakeLists.txt          <-- Projekt-Level CMake
├── sdkconfig               <-- Globale Projektkonfiguration (generiert)
├── sdkconfig.defaults      <-- Standardwerte auf denen sdkconfig basiert
├── components/             <-- (Optional) Eigene, wiederverwendbare Module
│   └── meine_bibliothek/
│       ├── CMakeLists.txt
│       ├── include/
│       │   └── meine_bib.h
│       └── meine_bib.c
├── main/                   <-- Der Hauptcode (wird als Komponente behandelt)
│   ├── CMakeLists.txt      <-- Komponenten-Level CMake
│   ├── main.c              <-- Einstiegspunkt (app_main)
│   ├── *.c, *.h, ...       <-- Alternative Modularisierung
│   └── Kconfig.projbuild   <-- eigene Parameter für sdkconfig
├── managed_components/     <-- Von Espressif gestellte Bibliotheken
│   ├── Komponente1/        <-- Verzeichnis mit den Daten von Komponente1
│   ├── ...                  <-- Weitere Komponenten
└── build/                  <-- (Generiert) Kompilierte Dateien, Binaries

```

2. Die Kern-Elemente im Detail

A. Das Herzstück: Die zwei CMakeLists.txt

Das häufigste Stolperperverse für Anfänger ist die Existenz von mehreren CMakeLists.txt Dateien. Sie haben völlig unterschiedliche Aufgaben:

i. Projekt-Level CMakeLists.txt (im Wurzelverzeichnis):

Diese Datei definiert das Projekt als Ganzes. Sie macht eigentlich nichts anderes, als das ESP-IDF Build-System zu laden und dem Projekt einen Namen zu geben.

ii. Komponenten-Level CMakeLists.txt (im main/ Ordner):

Hier passiert die eigentliche Arbeit. Diese Datei teilt dem Compiler mit, welche .c-Dateien kompiliert werden sollen und wo die zugehörigen Header (.h) liegen.

B. Die Steuerzentrale: sdkconfig

Die Datei sdkconfig ist das Gehirn des Projekts. Sie enthält Hunderte von Konfigurationsparametern für den ESP32 (z. B. WLAN-Passwörter, Taktfrequenzen, RTOS-Einstellungen, Flash-Größen). Die Regel: Diese Datei wird niemals von Hand im Texteditor bearbeitet. Sie wird erzeugt, sobald das Projekt das erste Mal kompiliert wird oder der folgende Befehl eingegeben wird. Alles was in der Datei sdkconfig.defaults vorgegeben ist, wird in die sdkconfig übernommen

```
idf.py fullclean
```

Dann wird das komplette IDF-Framework nach Kconfig Dateien durchsucht und deren Inhalt in die sdkconfig Datei geschrieben. Will man eigene Parameter hinzufügen, kann man im main-Verzeichnis eine Datei namens Kconfig.projbuild erstellen

Will man den Parametern Werte zuweisen, nutzt man das Kconfig-Menü über das Terminal:

```
idf.py menuconfig.
```

Das Tool generiert aus den grafischen (oder Terminal-basierten) Eingaben die sdkconfig Datei sowie einen C-Header (sdkconfig.h), der im gesamten Code über Makros verfügbar ist. Gleichzeitig kann man mit diesem Tool auch Werte den Konfigurationsparametern zuweisen

C. Der main/ Ordner

In Standard-C-Programmen gibt es eine main() Funktion. Im ESP-IDF, das auf dem Echtzeitbetriebssystem FreeRTOS basiert, heißt der Einstiegspunkt app_main().

	IT-Technik cyber-physische Systeme		Architektur Espressif IDF	
	Nachname, Vorname:		Klasse: IF11	Datum:

Sobald der ESP32 bootet, die Hardware initialisiert und das Betriebssystem gestartet hat, übergibt FreeRTOS die Kontrolle an einen Task, der diese `app_main()` ausführt.

D. Der managed_components/ Ordner

Dieser Ordner wird über den Component Manager verwaltet und enthält alle Komponenten, die von Espressif gepflegt werden und von dem Projekt benötigt werden. Am einfachsten fügt man sie über den folgenden Befehl hinzu:

```
idf.py add-dependency "espressif/esp_d1^1.0.0"
```

E. Der components/ Ordner (Für saubere Architektur)

Sobald ein Projekt wächst, gehört der Code nicht mehr in die `main.c`. Eigene Treiber (z. B. für einen I2C-Sensor) oder Algorithmen werden in den `components/` Ordner ausgelagert. Das ESP-IDF Build-System sucht beim Kompilieren automatisch in diesem Ordner. Jede Komponente dort benötigt lediglich wieder eine eigene, kleine `CMakeLists.txt`, um sich dem System bekannt zu machen. So lernt man direkt moderne, entkoppelte Softwarearchitektur.

F. Der build/ Ordner (Blackbox)

Sobald man `idf.py build` ausführt, wird dieser Ordner erstellt. CMake generiert hier die Ninja-Build-Dateien, der Compiler legt die Objekt-Dateien (`.o`) ab und der Linker baut am Ende die `.bin` und `.elf` Dateien zusammen, die auf den ESP geflasht werden.

Übung:

- Klone das Projekt <https://github.com/docblock/03ESP32p4webcamH264> in den source-Ordner
- Das Programm stellt eine Verbindung zu einem WLAN her. Die SSID und das Passwort werden in `sdkconfig` vorgegeben. Die Einstellungen in `menuconfig` findet man unter Example Configuration.
- Kompiliere das Projekt und übertrage es auf den ESP32.
- Die zugewiesene IP-Adresse findet man im seriellen Monitor. Rufe die IP-Adresse im Browser auf.